<ALGORITMA PEMROGRAMAN 1B>

PERTEMUAN KE 6 INPUT DARI KEYBOARD DAN OPERASI FILE

PROGRAM STUDI INFORMATIKA
UNIVERSITAS GUNADARMA

Cangkupan Materi Pertemuan 6

1

Input dari Keyboard dan Operasi File

- Input dari Keyboard
- Membuat, membuka file
- Errors dan Exception

1

Input dari Keyboard dan Operasi File

ALGORITMA PEMROGRAMAN 1B

PROGRAM STUDI INFORMATIKA
UNIVERSITAS GUNADARMA

Input dari Keyboard pada Python

Program yang selama ini kita buat, nilainya telah ditentukan sebelumnya, program - program tersebut tidak mendapatkan input dari user.

Program - program tersebut hanya melakukan hal yang sama setiap waktu.

Python menyediakan fungsi built-in yang mengambil nilai langsung dari input keyboard.

Fungsi yang paling sederhana dinamakan **input()**. Ketika fungsi ini dipanggil, program dihentikan dan menunggu masukan dari user untuk mengetikkan sesuatu. Pada saat user menekan **tombol Enter**, maka program tersebut dilanjutkan dan fungsi input mengembalikan apa yang user ketik, sebagai tipe data string.

Contoh :

```
>>> input = input()
silahkan ketik disini          # mengetik silahkan ketik disini, lalu tekan tombol enter
>>> print(input)
silahkan ketik disini
```

Input dari Keyboard pada Python

Sebelum memanggil fungsi input() , sebaiknya kita memasukkan parameter argumen sebagai prompt pada input:

Contoh:

```
>>> nama = input("Siapa nama anda?")
Siapa nama anda?santi                    # mengetik santi, lalu tekan tombol enter
>>> print(nama)
santi
```

Input dari Keyboard pada Python

Jika kita mengharapkan input yang akan dimasukan bertipe data integer, kita dapat membungkus input() memakai int() untuk bilangan bulat dan float() untuk bilangan desimal.

Contoh:

```
>>> angka = input("Masukkan sebuah angka: ")
Masukkan sebuah angka: 5  # mengetik angka 5
>>> print(type(angka))      # tipe data variabel angka masih string
<class 'str'>
>>> angka = int(angka)      # mengubah tipe data variabel angka dari string menjadi integer
>>> print(type(angka))      # tipe data variabel angka sudah menjadi integer
<class 'int'>
>>> print(angka)
5
```

Pengoperasian File pada Python

Python menyediakan fungsi built-in bernama open() untuk kebutuhan pengoperasian file, seperti membuat, membuka, maupun menulis file. bentuk umum dari fungsi open():

open(file, mode)

ket:

file : path dan nama sebuah file

mode : sebuah string yang menentukan mode yang akan dipakai

Pengoperasian File pada Python

Ada empat metode (mode) yang berbeda untuk membuka file:

- "r" - Read - **Nilai Default**. Membuka file untuk membaca, jika file tidak ada maka akan error.
- "a" - Append - Membuka file untuk ditambahkan, jika file tidak ada maka file akan dibuat
- "w" - Write - Membuka file untuk menulis, jika file tidak ada maka file akan dibuat
- "x" - Create - Membuat file yang ditentukan, jika file ada maka akan error

perbedaan mode "a" dengan "w" :

- "a" - Append - akan ditambahkan diakhir isi file
- "w" - Write - akan overwrite atau menimpa isi file yang ada

Pengoperasian File pada Python

beberapa metode pada objek file:

- metode read()

read() berfungsi untuk menelusuri isi file sampai akhir file dan apabila metode read() sudah mencapai akhir dari file (atau sudah menggunakan metode read() sebelumnya). contoh:

```
# f berisi file sample.txt yang memiliki isi konten:
# Hello
# World
>>> f = open ( “sample.txt", “r")
>>> f.read()
  ‘hello\nworld’
```

Pengoperasian File pada Python

- metode readline()

readline() berfungsi untuk membaca isi file perbaris atau sampai menemukan karakter ASCII "\n". Sama seperti halnya penggunaan read(). Metode readline() akan menampilkan string kosong jika sudah mencapai akhir dari file. Contohnya :

```
# f berisi file sample.txt yang memiliki isi konten:
# Hello
# World
>>> f = open ( “sample.txt",  “r")
>>> f.readline()
'Hello\n'
>>> f.readline()
'World'
>>> f.readline()
''
```

Pengoperasian File pada Python

- metode write(<string>)

write() berfungsi untuk memerlukan argumen parameter string, untuk dituliskan ke dalam file dan tidak menghasilkan tampilan hasil. Perhatikan metode file yang digunakan, metode r akan menimbulkan pesan kesalahan jika file dibuka dengan metode write. Contohnya :

```
>>> f = open ("tes.txt", "w")
>>> f.write("Tes untuk menulis ke dalam file")          # menulis pada file tes.txt
```

Pengoperasian File pada Python

- metode close()

close() berfungsi untuk untuk menutup file dan membebaskan sumber daya sistem yang digunakan secara langsung. Contohnya:

```
## file pengujian: sample1.txt
## isi file sample1.txt = Hello World, 12345.
>>> f = open("sample1.txt", "r")        # membuka file sample1.txt
>>> print(f.read())
Hello World, 12345.
>> f.close()                            # menutup file smaple1.txt
```

Pengoperasian File pada Python

Contoh Penggunaan "r" :

```
## file pengujian: sample1.txt
## isi file sample1.txt = Hello World, 12345.
>>> f = open("sample1.txt", "r")  # membuka file sample1.txt
>>> print(f.read())
Hello World, 12345.
```

Contoh penggunaan mode "x" :

```
>>> f = open("sample2.txt", "x") # akan membuat file kosong sample2.txt
```

Pengoperasian File pada Python

Contoh penggunaan mode "a" :

```
## file pengujian: sample3.txt
## isi file sample13.txt:
## ini baris pertama
## ini baris kedua
>>> f = open("sample3.txt", "a")        # membuka file sample3.txt dengan mode append
>>> f.write("ini diketik dengan mode 'a' ")   # menambahkan isi pada file sample3.txt
>>> f.close()
>>> f = open("sample4.txt", "r")                # membuka file sample3.txt dengan mode read
>>> print(f.read())                             # isi file sample3.txt
ini baris pertama
```

Pengoperasian File pada Python

Contoh penggunaan mode "w" :

```
## file pengujian: sample4.txt
## isi file sample14.txt:
## ini baris pertama
## ini baris kedua
>>> f = open("sample4.txt", "w")                # membuka file sample4.txt dengan mode read write
>>> f.write("ini diketik dengan mode 'w' ")     # menimpa isi pada file sample4.txt
>>> f.close()
>>> f = open("sample4.txt", "r")                # membuka file sample4.txt dengan mode read read
>>> print(f.read())                             # isi file sample4.txt
ini diketik dengan mode 'w'
```

2

Errors dan Exception pada Python

ALGORITMA PEMROGRAMAN 1B

PROGRAM STUDI INFORMATIKA
UNIVERSITAS GUNADARMA

Errors dan Exception pada Python

Pemograman komputer adalah sebuah proses yang sangat kompleks, dan karena hal ini dilakukan manusia maka seringkali terjadi kesalahan - kesalahan. Pada umumnya kesalahan - kesalahan dalam pemograman dikenal sebagai bugs dan proses penelusuran program kembali dan memperbaikinya dikenal dengan debugging.

Tiga jenis kesalahan dapat terjadi dalam sebuah program:

- Kesalahan Sintaks
- Runtime Errors
- Kesalahan Algoritma

Errors dan Exception pada Python - Kesalahan Sintaks

Python hanya dapat mengeksekusi program tersebut hanya jika program tersebut berisi baris - baris perintah dengan sintaks yang benar. Kalau dalam program - program tersebut terdapat kesalahan sintaks maka proses akan berhenti dan menampilkan pesan - pesan kesalahan, yang kemudian dikenal sebagai Syntax errors.

Kesalahan sintaks, dapat juga disebut dengan kesalahan dalam memparsing kode python yang salah, umumnya ditemui pada saat Anda baru memulai belajar bahasa pemograman python. Contohnya :

```
>>> while 1 print 'hello world'
  File "<stdin>", line 1
    while 1 print 'hello world'
            ^
SyntaxError: invalid syntax
```

Errors dan Exception pada Python - Runtime Errors

- Jenis kesalahan yang kedua disebut dengan runtime errors, disebut begitu karena kesalahan tidak akan muncul sampai Anda menjalankan program tersebut. Kesalahan ini juga dikenal dengan exceptions atau pengecualian karena biasanya mengindikasikan sesuatu pengecualian yang buruk telah terjadi.

- Runtime errors sangat jarang terjadi pada program - program yang sederhana.

Errors dan Exception pada Python - Kesalahan Algoritma

Jenis kesalahan selanjutnya adalah kesalahan algoritma, yang kemudian dikenal dengan **semantic errors**. Jika terdapat kesalahan jenis ini dalam program Anda, program Anda akan berjalan dan tidak mengeluarkan pesan - pesan kesalahan, tetapi tidak akan sesuai dengan harapan Anda. Akan terjadi penyimpangan dari keinginan Anda.

Karena program tersebut tidak sesuai dengan harapan Anda dan akan meminta Anda untuk menelusuri kembali program tersebut dari awal untuk memperbaiki algoritmanya, kesalahan ini akan sering muncul pada saat Anda mulai berpengalaman dengan suatu bahasa pemograman.

Errors dan Exception pada Python - Exception

Jika terjadi kesalahan pada saat program dijalankan (run-time errors), program tersebut membuat sebuah pengecualian (exceptions). Biasanya program terhenti dan menampilkan pesan kesalahan. Contohnya pada pembagian bilangan dengan nol :

```
>>> 40 / 0
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ZeroDivisionError: division by zero
```

Errors dan Exception pada Python - Exception

Contoh lainnnya adalah pengaksesan yang bukan elemen anggota dari suatu list, seperti:

```
>>> a = []
>>> a[5]
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
IndexError: list index out of range
```

Errors dan Exception pada Python - Handling Exception

Kadang - kadang kita ingin menjalankan sebuah operasi yang dapat menyebabkan kesalahan pada runtime, tetapi kita tidak mau program tersebut berhenti total. Kita dapat mengatasinya dengan perintah try dan except. Perintah try menjalankan perintah pada blok pertama. Jika tidak ada kesalahan yang terjadi, perintah except akan diabaikan. Tetapi jika terjadi kesalahan apapun jenisnya, program akan menjalankan perintah - perintah pada ruang lingkup except dan kemudian program tersebut berlanjut.

Contohnya:

```
def membaca_file(namafile):
try :
        f = open(namafile, 'r')
        f.close()
        return 1            # mengembalikan nilai 1 karena berhasil membaca file.
except :
        return 0            # mengembalikan nilai 0 karena terdapat error statement pada blok try
```

Errors dan Exception pada Python - Handling Exception

perintah try dan except memiliki klausa opsional, yaitu **else**. blok else akan berjalan ketika kode yang dieksekusi pada blok try tidak ada error atau kesalahan. Contohnya:

```
>>> try:
...    print("Hello")
... except:
...    print("Error)
... else:
...    print("World”  )
...
Hello
World
```

Errors dan Exception pada Python - Handling Exception

perintah try dan except memiliki klausa opsional lagi, yaitu **finally**. Blok finally akan pasti dijalankan meskipun kode pada blok try mengalami error maupun tidak. Contohnya:

```
>>> try:
...     print(x)
... except:
...     print("Error")
... finally:
...     print("Perintah try-except telah dijalankan")
...
Error
Perintah try-except telah dijalankan
```

Daftar Pustaka

- https://www.w3schools.com/python/default.asp
- https://docs.python.org/3.9/tutorial/index.html
- http://dini3asa.staff.gunadarma.ac.id/Downloads/files/19689/INPUT-OPFILE-ERROREXCEP-fix.pdf
  - https://drive.google.com/drive/folders/1XxWPQJzdiuaq_nX18uglGAPx5nLUlchJ?usp=sharing
    - Guido van Rossum. 2003. Python Tutorial. PythonLabs
    - Bob Dowling. An introduction to Python for absolute beginners. University Computing Service
    - Mark Lutz. 2009. Powerfull Object Oriented Programming : Learning Python 4th Edition. O' Reily.

Tugas

KERJAKAN TUGAS DIBAWAH INI, LALU UPLOAD KEMBALI JAWABAN PROGRAMNYA DENGAN NAMA FILE NAMA_NPM_M6-AP1B.DOC

Buatlah sebuah exception yang mengharuskan untuk memasukkan nilai minus (-1) dan menampilkan pesan kesalahan (exceptions) jika yang dimasukkan bukanlah tipe data bilangan integer atau float.

TERIMA KASIH

PROGRAM STUDI INFORMATIKA
UNIVERSITAS GUNADARMA

Tim Penyusun:
1. Dini Triasanti
2.
3